

Assignment: Inference Engineering Week Task

Mix: theory + coding, focused on implementation and optimization basics **Note:** everything can run on CPU on your personal system

Part A Theory

1. What's the difference between latency and throughput in model inference, which factor will influence model's performance more and how?
2. Explain what KV cache is and how it helps with model's generation.
3. What is quantization, and what's the trade-off between int8/int4 quantization and model accuracy? Also give an example with calculations and formula of how quantization can be performed.
4. Name one real inference serving framework (like vLLM or TensorRT-LLM) and briefly describe one optimization it does.

Part B Programming (Python, CPU only, keep it simple)

Task 1: Measure latency vs batch size

- Load a small pretrained model on CPU (e.g. `distilbert-base-uncased` or `distilgpt2` from HuggingFace, both are light enough for CPU)
- Run inference with batch sizes 1, 2, 4, 8 (kept small since it's CPU)
- Measure and print time taken per batch
- Plot batch size vs total time, and batch size vs time-per-sample
- Write 2-3 lines on what pattern they observe

Task 2: Implement basic quantization (CPU friendly)

- Load the same small model
- Quantize it using `torch.quantization.quantize_dynamic` (this works on CPU, no GPU needed)
- Compare model size before/after quantization (use `torch.save` and check file size, or `sys.getsizeof` on state dict)
- Run a few sample inputs and compare inference time before vs after
- Write 2-3 lines on the speed/size difference observed

Submission

- Runtime should be under 10-15 mins total on a normal laptop, no GPU

Format for submission

- Theory — write it up in LaTeX, compile to PDF, submit the PDF (Part A only)
- Code — one Jupyter notebook (.ipynb) for Part B
 - all outputs and plots should run and be visible in the notebook cells
 - make sure to push on github
- Submit : repo link with latex and it's pdf; as well as .ipynb notebook and its exported pdf